Time Series Forecasting of S&P 500 Returns: A Comparative Study of AR, ARMA, GARCH, and LSTM Models

Jiawei GUO - 2250019302

Abstract

The Standard & Poor's 500 Index (S&P 500) serves as a representative benchmark of the U.S. stock market. This study examines daily closing prices of the S&P 500 from January 3, 1980, to August 31, 2010, constructing time series forecasting models based on log return sequences. We first verify stationarity through unit root tests and conduct preliminary model order identification using ACF, PACF, and EACF. Subsequently, we compare and select between AR and ARMA models using AIC and BIC information criteria to determine appropriate mean model structures.

After establishing mean models, residual diagnostics through Ljung-Box and ARCH-LM tests reveal that while residuals show no significant autocorrelation, they exhibit clear conditional heteroscedasticity. To capture the volatility clustering phenomenon prevalent in financial time series, we introduce GARCH(1,1) models to the mean equations, employing Student-t distributions for error terms, thereby constructing AR-GARCH and ARMA-GARCH models. Further residual diagnostics confirm the effective correction of heteroscedasticity by volatility models.

Building on this foundation, we compare traditional statistical models with LSTM neural networks, evaluating both in-sample fit and out-of-sample forecasting performance. Results indicate that while prediction errors are comparable across models for point forecasts, GARCH-class models demonstrate clear advantages in volatility characterization and risk description. The LSTM model, under our research settings, does not exhibit significantly superior forecasting ability compared to traditional models. This study systematically summarizes the complete workflow of financial time series modeling, providing a reference empirical framework for related forecasting and risk analysis.

Keywords: Time Series Forecasting, S&P 500, AR Model, ARMA Model, GARCH Model, LSTM, Conditional Heteroscedasticity, Volatility Clustering

1. Introduction

Time series forecasting is a fundamental research area in statistics and econometrics, with extensive applications in financial market analysis, risk management, and asset allocation. Stock market price series, as typical economic time series, are characterized by strong randomness and significant volatility, making their modeling and forecasting core challenges in financial time series analysis. The S&P 500 index, established in 1957, tracks 500 common stocks representing approximately 80% of total U.S. market capitalization. Developed and maintained by S&P Dow Jones Indices LLC (a joint venture controlled by S&P Global), the S&P 500 possesses sufficient representativeness to reflect the rise and fall of the U.S. economy, making it a commonly used subject in financial time series modeling research (from Wikipedia, 2025).

Traditional linear time series models, such as Autoregressive (AR) and Autoregressive Moving Average (ARMA) models, have clear theoretical foundations and good interpretability in characterizing linear correlation structures of stationary time series. These models typically identify model orders through Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF), combined with information criteria for model selection, and are widely applied in economic and financial time series analysis. However, substantial empirical research demonstrates that financial asset return series often exhibit volatility clustering and conditional heteroscedasticity, and linear models relying solely on mean structure modeling struggle to adequately characterize their statistical properties.

To address these issues, Generalized Autoregressive Conditional Heteroskedasticity (GARCH) models were proposed to effectively describe common volatility change behavior in financial time series through dynamic modeling of conditional variance. GARCH models have become important tools in financial econometric analysis, with advantages in characterizing time-varying risk features while maintaining linear mean structures.

In recent years, with enhanced computational capabilities and the development of machine learning methods, deep learning models have been gradually introduced to time series forecasting. Among these, Long Short-Term Memory (LSTM) networks, as a special recurrent neural network structure, can capture nonlinear relationships and long-term dependency features in time series to some extent. However, the actual advantages of deep learning models in financial time series forecasting remain controversial, and whether their forecasting performance significantly exceeds traditional statistical models requires comparative analysis within a unified experimental framework.

Based on this background, this study examines daily closing prices of the S&P 500 from January 3, 1980, to August 31, 2010, constructing time series forecasting models through log return sequences. We sequentially employ AR, ARMA, and models incorporating GARCH structures to model the data, with model correction and verification through residual diagnostics. Building on this foundation, we introduce LSTM models as a comparative method to systematically compare in-sample fit and out-of-sample forecasting performance across different models. This study aims to organize the standard process of financial time series modeling and analyze differences in forecasting ability and model interpretability across different methods.

2. Methodology

2.1 Stationarity and Model Framework

Time series model construction typically requires stationarity as a prerequisite. For financial price series, level values often exhibit non-stationary processes, while log return series empirically satisfy weak stationarity conditions. Therefore, this study uses log returns as the modeling object to ensure reasonable model assumptions.

Building on this foundation, we first employ linear time series models to characterize the conditional mean structure of returns, verifying model assumptions through residual diagnostics. When residuals exhibit conditional heteroscedasticity, we introduce volatility models to model variance, thereby progressively refining the model structure.

2.2 Autoregressive (AR) Model

The autoregressive model assumes that the current value of a time series can be represented by a linear combination of

its past values. Let r_t represent the log return at time t, then the AR(p) model can be expressed as:

$$r_t = \mu + \sum_{i=1}^{p} \phi_i r_{t-i} + \varepsilon_t,$$

where μ is the constant term, ϕ_i are autoregressive coefficients, and ε_t is white noise disturbance with zero mean and constant variance.

The AR model is suitable for stationary time series and is primarily used to capture linear dependence structures. In practical modeling, model orders are preliminarily identified through ACF and PACF, followed by selection based on information criteria.

2.3 Autoregressive Moving Average (ARMA) Model

The ARMA model extends the AR framework by incorporating a moving average component to account for serial correlation in the error terms. An ARMA(p, q) model is given by:

$$r_t = \mu + \sum_{i=1}^{p} \phi_i r_{t-i} + \sum_{j=1}^{q} \theta_j \epsilon_{t-j} + \varepsilon_t,$$

where θ_j denote the moving average coefficients. By jointly modeling the autoregressive and moving average components, the ARMA model provides greater flexibility in capturing short-term dynamics of stationary time series. However, it still assumes constant error variance.

2.4 Generalized Autoregressive Conditional Heteroscedasticity (GARCH) Model

Empirical studies of financial returns have consistently documented the presence of conditional heteroskedasticity, where volatility varies over time. The ARCH model proposed by Engle (1982) and its generalization, the GARCH model developed by Bollerslev (1986), provide a systematic statistical framework for modeling this phenomenon (Tsay, 2014). Let $\varepsilon_t = r_t - \mu_t$ be the innovation at time t. In the GARCH(1, 1) specification, the error term is decomposed as:

$$\varepsilon_t = \sigma_t z_t, \quad z_t \sim i.i.d.(0, 1),$$

with the conditional variance evolving according to:

$$\sigma_t^2 = \alpha_0 + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2,$$

where $\alpha_0 > 0$, $\alpha_1 \geq 0$, and $\beta_1 \geq 0$. And the condition $\alpha_1 + \beta_1 < 1$ is typically imposed to ensure a finite unconditional variance of ε_t. The GARCH model is typically combined with an AR or ARMA mean equation, allowing volatility dynamics to be modeled without altering the conditional mean structure. This framework allows for a more realistic representation of volatility dynamics while preserving a linear structure in the conditional mean. To better accommodate the heavy-tailed nature of financial returns, this study assumes that the innovation term follows a Student-t distribution.

2.5 Model Estimation and Diagnostic Principles

Model parameters are estimated using maximum likelihood estimation. After estimation, residual diagnostic tests are conducted to assess the validity of the model assumptions. Specifically, the Ljung–Box test is applied to examine residual autocorrelation, while the ARCH-LM (Lagrange Multiplier) test is used to detect remaining conditional heteroskedasticity (Tsay, 2014). Residual distribution plots are also examined to assess model assumptions. If the residuals fail to satisfy the white noise assumption, the model is revised accordingly. This iterative process continues until a statistically adequate final model is obtained.

2.6 Long Short-Term Memory (LSTM) Model

The LSTM network (Hochreiter & Schmidhuber, 1997) is an advanced recurrent neural network architecture designed to mitigate the vanishing gradient problem through the use of gating mechanisms. Unlike traditional statistical models, LSTM

does not rely on explicit distributional assumptions. Instead, it learns a nonlinear mapping between input and output sequences directly from data. In this study, the LSTM model takes the logarithmic return series as input and adopts a one-step-ahead forecasting strategy. The prediction can be expressed as:

$$\hat{r}_t = f_{LSTM}(r_{t-1};\ \Theta)$$

where Θ denotes the network parameters. Due to the lack of explicit statistical interpretation, the LSTM model is primarily evaluated based on forecasting performance rather than structural inference.

3. Experimental Design

3.1 Data Preprocessing

- **Data Transformation:** To ensure statistical stationarity and enhance interpretability, we convert raw closing prices (P_t) to log returns (r_t):

$$r_t = \ln(P_t) - \ln(P_{t-1})$$

- **Data Split:** Following temporal order, we divide data into training (80%) and testing (20%) sets. All models use identical splits to ensure fair comparison.

3.2 Model Estimation Procedure

3.2.1 Econometric Models (AR, ARMA, GARCH)

Estimation Steps:

1. **Stationarity Testing:** Apply Augmented Dickey-Fuller (ADF) to ensure r_t follows $I(0)$ process;
2. **Mean Equation Selection:** Determine optimal ARMA(p, q) structure by minimizing Akaike Information Criterion (AIC);
3. **Variance Equation Specification:** Fit GARCH(1,1) parameters using Maximum Likelihood Estimation (MLE) based on mean equation residuals;
4. **Distributional Assumption:** Explicitly compare Normal vs. Student-t innovations to handle leptokurtosis.

3.2.2 Deep Learning Model (LSTM)

Unlike return-based econometric models, LSTM is scale-sensitive. Therefore, preprocessing is necessary:

- MinMaxScaler Normalization: Data scaled to [0,1] range: $x' = \frac{x_t - x_{min}}{x_{max} - x_{min}}$
- Windowing: Problem structured as supervised learning using input $X = \{x_{t-\omega}, \dots, x_{t-1}\}$ to predict $Y = \{x_t\}$. Here we set window size set to $\omega = 60$.

Hyperparameters:

- Architecture: Stacked LSTM layers (50 units + 50 units) followed by Dense layer;
- Regularization: Dropout (rate=0.2) to prevent overfitting;
- Optimization: Adam optimizer with Mean Squared Error (MSE) loss.

3.3 Evaluation Metrics

To provide robust comparison, we employ the following metrics for out-of-sample performance:

- Mean Squared Error (MSE): $MSE = \frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y}_t)^2$. Heavily penalizes large forecast errors
- Mean Absolute Error (MAE): $MAE = \frac{1}{n}\sum_{t=1}^{n}|y_t - \hat{y}_t|$. Provides linear scoring of error magnitude
- Root Mean Square Error (RMSE): $RMSE = \sqrt{MSE}$
- Residual Diagnostics: Post-estimation analysis using Ljung-Box test for autocorrelation and ARCH-LM test for residual heteroscedasticity

4. Experiment

4.1 Data Preparation

This study employs daily closing prices of the S&P 500 index from January 3, 1980 to August 31, 2010, comprising a total of 7,736 observations. Figure 1 illustrates the raw price series together with its logarithmic transformation. While the log transformation mitigates part of the heteroskedasticity observed in the raw series, a pronounced upward trend remains evident. Figure 2 further presents the autocorrelation functions (ACF) of both the raw and log-transformed price series. In both cases, autocorrelations remain close to unity even at large lags, indicating strong persistence and non-stationarity.

To formally assess stationarity, Augmented Dickey–Fuller (ADF) unit root tests are conducted on both series, with results summarized in Table 1. The corresponding p-values are well above conventional significance levels, implying failure to reject the unit root null hypothesis and confirming non-stationarity in levels.

To eliminate non-stationarity, the logarithmic price series is first-differenced to obtain the log-return series (Figure 3). ADF test results indicate that the log returns are stationary at the 1% significance level, satisfying the key assumption required for subsequent time series modeling.

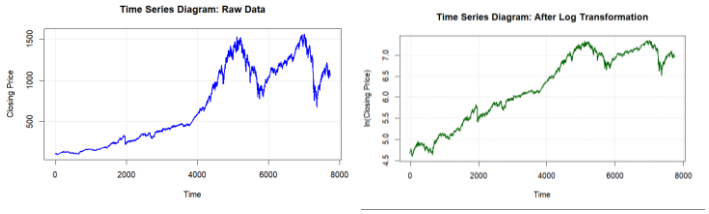


Figure 1: Raw S&P 500 Daily Closing Prices and Log-Transformed Series

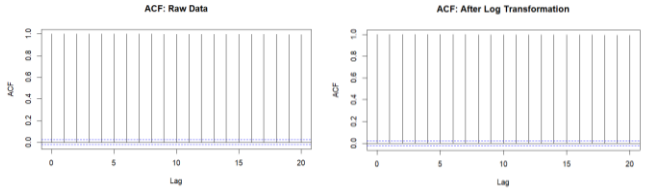


Figure 2: Autocorrelation Functions of Raw and Log-Transformed Price Series

Type	Dickey-Fuller statistic	p-value
Raw data	0.2943	0.7101
Logarithmic data	2.0564	0.99
Log return	-66.2506	0.01

Table 1: Augmented Dickey–Fuller (ADF) Unit Root Test Results

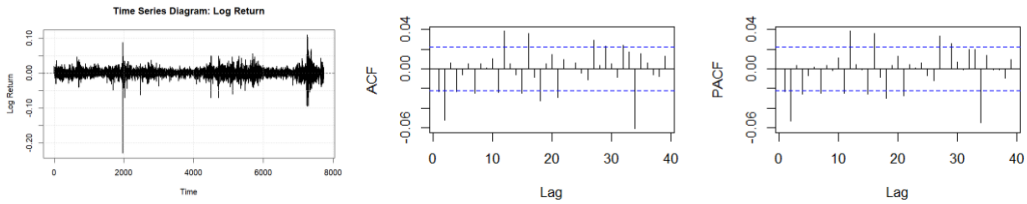


Figure 3: Log-Return Series of the S&P 500 Index

4.2 AR Model

4.2.1 Model Estimation

After establishing the stationarity of the log-return series, an AR model is first employed to characterize its linear mean dynamics. Figure 4 presents the ACF and PACF of the log returns. Neither plot exhibits a clear cutoff or decay pattern, making it difficult to determine the model order solely based on visual inspection. Consequently, an automatic order selection procedure based on the AIC is applied to identify the optimal AR specification. The results indicate that the AR(4) model achieves the lowest AIC value and is therefore selected as the preferred mean equation.

The estimated AR(4) model is given by:

$$r_t = 0.0004 + 0.0211r_{t-1} - 0.0321r_{t-2} - 0.0287r_{t-3} - 0.0193r_{t-4} + \varepsilon_t$$

Parameter estimates, along with their standard errors, t-statistics, and p-values, are reported in Table 2. Several lagged coefficients are statistically significant at conventional levels, indicating the presence of short-term linear dependence in the return series.

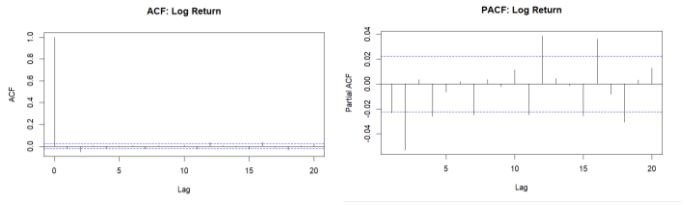


Figure 4: ACF and PACF of the Log-Return Series

Parameter	Estimate	Std. Error	t-value	p-value
μ (intercept)	0.0004	0.0001	2.97	0.003
ϕ_1	0.0211	0.0127	1.66	0.097
ϕ_2	-0.0321	0.0127	-2.52	0.011
ϕ_3	-0.0287	0.0127	-2.26	0.024
ϕ_4	-0.0193	0.0127	-1.52	0.128
$\sigma^2 = 0.0001133$; Log-likelihood = 19330.01; AIC = -38648.03				

Table 2: Parameter Estimates of the AR(4) Model

4.2.2 Diagnostic Results

Following model estimation, a series of diagnostic checks is conducted on the residuals of the AR(4) model. The Ljung–Box test yields a p-value of 0.7517 across multiple lags, indicating no evidence of residual autocorrelation and suggesting that the AR(4) model adequately captures the linear dependence in the mean. However, further diagnostics reveal notable shortcomings. The QQ plot of residuals (Figure 5) shows substantial deviations in both tails, implying skewness and heavy-tailed behavior. Moreover, the ARCH-LM test strongly rejects the null hypothesis of no ARCH effects (p-value $< 2.2\times10^{-16}$), indicating pronounced conditional heteroskedasticity in the residuals. These findings suggest that while the AR(4) model is appropriate for modeling the conditional mean, its assumption of constant variance is violated. As a result, the AR model alone is insufficient to fully describe the dynamics of financial return series.

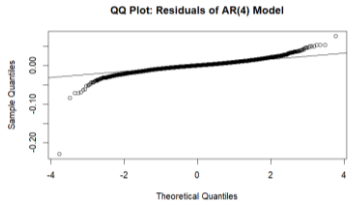


Figure 5: QQ Plot of Residuals from the AR(4) Model

4.2.3 Performance Metrics

Figure 6 presents the out-of-sample forecasts generated by the AR(4) model, with corresponding performance metrics reported in Table 3. The relatively low values of MSE, RMSE, and MAE indicate that the AR(4) model provides a reasonable baseline for point forecasting of returns. It should be noted, however, that these results are obtained under the assumption of constant variance. Consequently, while the model performs adequately in terms of point forecasts, it lacks the ability to capture time-varying volatility and associated risk dynamics.

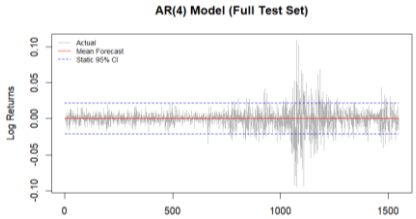


Figure 6: Test Set Forecast of the AR(4) Model

Metric	In-Sample	Out-of-Sample
AIC	-38648.03	-
MSE	-	0.000207884
RMSE	-	0.014418
MAE	-	0.008986

Table 3: Forecast Performance Metrics of the AR(4) Model

4.3 AR(4)-GARCH(1,1) Model

4.3.1 Model Specification

To address the conditional heteroskedasticity detected in the previous section, a GARCH(1,1) structure is incorporated into the AR(4) mean equation. In addition, the innovations are assumed to follow a Student-t distribution to better capture the heavy-tailed nature of financial returns. The mean and variance equations are specified as:

$$r_t = 0.000548 + 0.0228 \cdot r_{t-1} - 0.0203 \cdot r_{t-2} - 0.0389 \cdot r_{t-3} - 0.0221 \cdot r_{t-4} + \varepsilon_t,$$

$$\sigma_t^2 = 1e^{-6} + 0.0476 \cdot \varepsilon_{t-1}^2 + 0.9475 \cdot \sigma_{t-1}^2$$

Estimation results indicate a highly persistent volatility process, as reflected by the large GARCH coefficient ($\beta_1 = 0.9475$). The sum $\alpha_1 + \beta_1 = 0.9951$ is close to unity, implying slow decay of volatility shocks and long memory behavior. Furthermore, the estimated degrees of freedom (approximately 6.5) confirm the presence of heavy tails in the return distribution.

4.3.2 Advanced Diagnostics

After incorporating the GARCH structure, residual diagnostics show substantial improvement. Neither the standardized residuals nor their squared values exhibit significant autocorrelation across multiple lags. Furthermore, the p-values of the weighted ARCH-LM test were high under different lag settings (Lag 3: p = 0.1551; Lag 5: p = 0.4364; Lag 7: p = 0.6567), failing to reject the null hypothesis of no residual ARCH effect. These results confirm that the GARCH(1,1) specification successfully captures volatility clustering, allowing the model to satisfy statistical assumptions for both the conditional mean and variance. This finding is consistent with established results in financial time series theory.

4.3.3 Performance Metrics

Figure 7 reports the out-of-sample forecasts of the AR(4)-GARCH(1,1) model, with performance metrics summarized in Table 4. Although the MSE is similar to that of the AR(4) model, the GARCH model provides a dynamic confidence band. During periods of market stress, the confidence band of the GARCH model widens, providing a more realistic risk assessment than the fixed confidence band of the AR model.

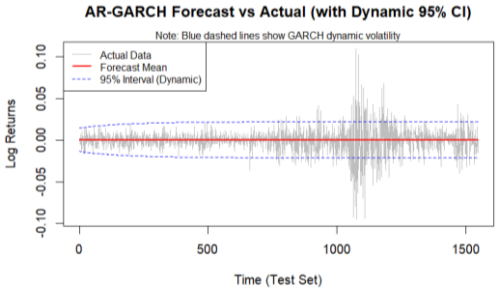


Figure 7: Out-of-Sample Forecast of the AR(4)-GARCH(1,1) Model

Metric	In-Sample	Out-of-Sample
AIC	-6.5791	-
MSE	-	0.000208051
RMSE	-	0.014424
MAE	-	0.008980

Table 4: Forecast Performance Metrics of the AR(4)-GARCH(1,1) Model

4.4 ARMA Model

4.4.1 Model Estimation

Building upon the AR framework, a moving average component is introduced to allow for a more flexible linear mean structure. The *auto.arima* function is employed to select the optimal model order based on AIC, resulting in an ARMA(2,1) specification. The estimated ARMA(2,1) equation is given by:

$$r_t = 0.0004 + 0.7092 \cdot r_{t-1} - 0.0494 \cdot r_{t-2} - 0.6880 \cdot \varepsilon_{t-1} + \varepsilon_t$$

Parameter estimates are reported in Table 5, where all coefficients are statistically significant at conventional levels. Nonetheless, a closer inspection reveals that the AR(1) coefficient (0.7092) and the MA(1) coefficient (−0.6880) are nearly equal in magnitude but opposite in sign. Such offsetting behavior is often indicative of parameter redundancy, suggesting that the additional MA term may not provide meaningful improvements over a simpler AR specification.

Parameter	Estimate	Std. Error	t-value	p-value
μ（intercept）	0.0004	0.0001	3.11	0.0019
ϕ_1	0.7092	0.1421	4.99	<0.001
ϕ_2	-0.0494	0.0131	-3.77	<0.001
θ_1	-0.6880	0.1421	-4.84	<0.001

Table 5: Parameter Estimates of the ARMA(2,1) Model

4.4.2 Diagnostic Results

Residual diagnostics for the ARMA(2,1) model indicate that linear dependence in the mean has been effectively removed. The Ljung–Box test yields a p-value of 0.8572, suggesting that the residuals are consistent with white noise. However, the QQ plot (Figure 8) exhibits substantial tail deviations, implying non-normality. Moreover, the ARCH-LM test strongly rejects the null hypothesis of homoskedasticity, indicating the persistence of conditional heteroskedasticity. These findings demonstrate that increasing the complexity of the mean equation alone is insufficient to address heteroskedasticity in financial return series, reinforcing the need for explicit volatility modeling.

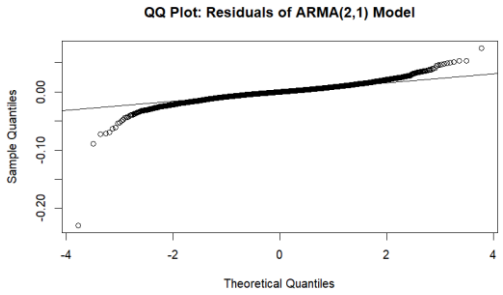


Figure 8: QQ Plot of Residuals from the ARMA(2,1) Model

4.4.3 Performance Metrics

Figure 9 presents the out-of-sample forecasts of the ARMA(2,1) model, with performance metrics summarized in Table 6. The results show that the out-of-sample MSE, RMSE, and MAE are nearly identical to those of the AR(4) model. According to Occam's Razor (from Wikipedia, 2025), the inclusion of additional parameters is not justified when predictive performance does not materially improve. Combined with evidence of parameter redundancy and heteroskedastic residuals, the ARMA(2,1) model offers no clear advantage over the simpler AR specification.

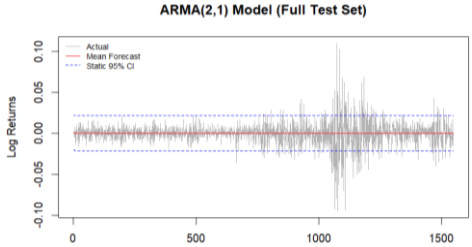


Figure 9: Out-of-Sample Forecast of the ARMA(2,1) Model

Metric	In-Sample	Out-of-Sample
AIC	-38649.97	-
MSE	-	0.000207892
RMSE	-	0.014418
MAE	-	0.008986

Table 6: Forecast Performance Metrics of the ARMA(2,1) Model

4.5 ARMA(2,1)-GARCH(1,1) Model

4.5.1 Model Specification

To jointly model the dynamics of the conditional mean and variance, the ARMA(2,1) mean equation is combined with a GARCH(1,1) variance specification, with innovations assumed to follow a Student-t distribution. The model is specified as:

$$r_t = 0.000554 + 0.7949 \cdot r_{t-1} - 0.0520 \cdot r_{t-2} - 0.7727 \cdot \varepsilon_{t-1} + \varepsilon_t$$

$$\sigma_t^2 = 1e^{-6} + 0.0472 \cdot \varepsilon_{t-1}^2 + 0.9480 \cdot \sigma_{t-1}^2$$

The variance parameters indicate strong volatility persistence ($\beta_1 \approx 0.948$), consistent with the AR-GARCH model and suggesting that volatility dynamics are primarily governed by the GARCH structure rather than increased mean equation complexity.

4.5.2 Advanced Diagnostics

While the ARMA-GARCH model successfully filters volatility clustering and passes weighted ARCH-LM tests (Lag3: p = 0.1518; Lag5: p = 0.4210; Lag7: p = 0.6399), its residual autocorrelation diagnostics are slightly inferior to those of the AR(4)-GARCH(1,1) model. See the Weighted Ljung-Box Test (Standardized Residuals) results:

- ARMA-GARCH: Lag 1: p = 0.0155; Lag 8: p = 4.441e-16; Lag 14: p = 0.005.
- AR-GARCH: Lag 1: p = 0.0184; Lag 11: p = 2.347e-09.

This discrepancy is likely attributable to parameter redundancy in the ARMA mean equation, which may undermine overall model stability. The results suggest that once volatility dynamics are adequately modeled, additional complexity in the mean equation does not necessarily improve model performance.

4.5.3 Performance Metrics

Figure 10 reports the out-of-sample forecasts of the ARMA(2,1)-GARCH(1,1) model, with performance metrics summarized in Table 7. Compared with the AR-GARCH model, out-of-sample forecast errors are nearly identical, while residual diagnostics are slightly weaker. Taking into account predictive accuracy, model stability, and parsimony, the AR(4)-GARCH(1,1) model is therefore preferred in this study.

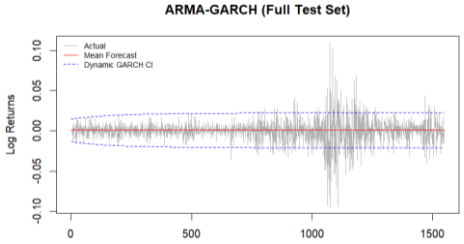


Figure 10: Out-of-Sample Forecast of the ARMA(2,1)-GARCH(1,1) Model

Metric	In-Sample	Out-of-Sample
AIC	-6.5798	-
MSE	-	0.000208064
RMSE	-	0.014424
MAE	-	0.008980

Table 7: Forecast Performance Metrics of the ARMA(2,1)-GARCH(1,1) Model

4.6 LSTM Model

4.6.1 Model Specification and Performance

To compare traditional econometric models with nonparametric approaches, a Long Short-Term Memory (LSTM) network is constructed for return prediction. The model employs a rolling window of 60 observations to capture temporal dependencies. The network structure, as shown in Figure 11, consists of two stacked LSTM layers (50 units each), regularized using Dropout (0.2), and the predicted values are finally output by a fully connected layer. Compared to the AR-GARCH model, which contains only a small number of parameters, this LSTM model contains 30,651 trainable parameters, resulting in significantly higher model complexity.

Table 8 summarizes the prediction performance metrics of the LSTM model. It can be seen that its out-of-sample error is slightly higher than that of traditional statistical models, and the AIC value is abnormally high. This phenomenon is not a direct reflection of poor model performance, but rather stems from the failure of the information criterion's penalty term for the number of parameters in high-dimensional neural networks.

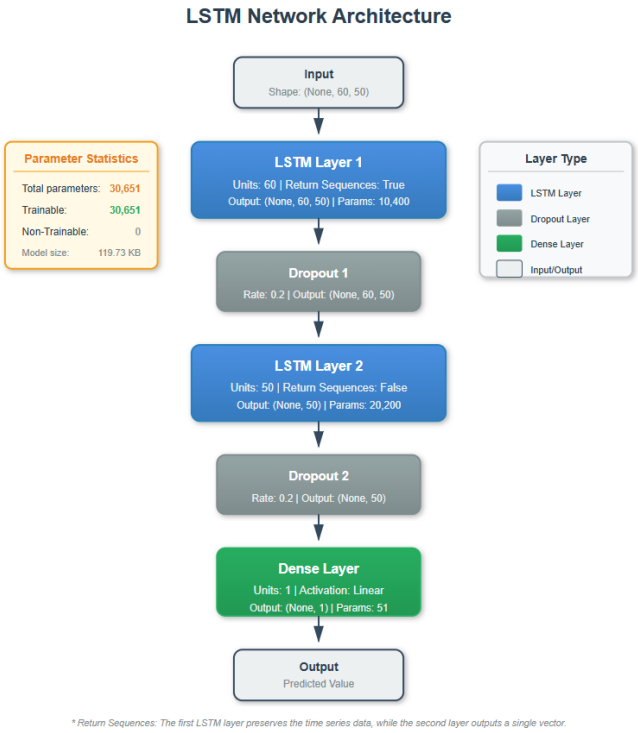


Figure 11: Architecture of the LSTM Neural Network

Metric	In-Sample	Out-of-Sample
AIC	5579.99	-
MSE	0.000114462	0.000210219
RMSE	0.010698681	0.014498940
MAE	0.007390203	0.009077944

Table 8: Forecast Performance Metrics of the LSTM Model

4.6.3 Visual Diagnostics Analysis

Figure 12 illustrates the training dynamics of the LSTM model. The training loss decreases rapidly in early epochs and stabilizes thereafter, while the validation loss closely follows, indicating that overfitting is reasonably controlled through regularization. Additionally, Figures 13 and 14 visualize the out-of-sample predictions for the full test set and the last 200 trading days, respectively. The model effectively tracks the central tendency of returns and captures mean-reversion and volatility clustering. However, extreme observations are consistently underestimated, reflecting a well-known shrinkage effect in neural networks optimized for MSE minimization.

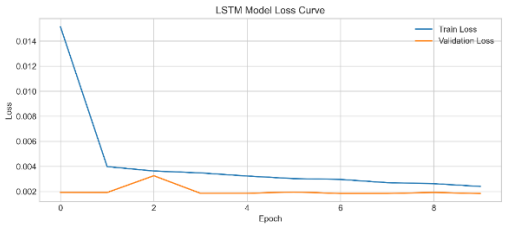


Figure 12: Training and Validation Loss Curves of the LSTM Model

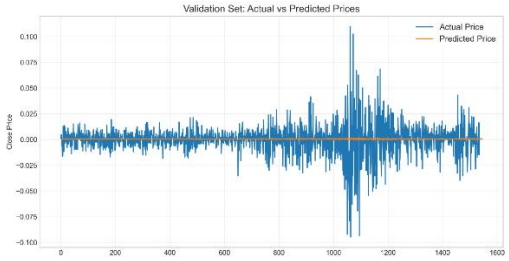


Figure 13: Out-of-Sample Predictions versus Actual Returns (LSTM)

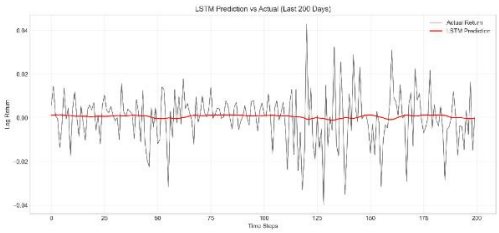


Figure 14: Comparison of Predicted and Actual Returns over the Last 200 Trading Days

5. Conclusions

This study systematically compares the modeling and forecasting performance of five classes of models—AR, ARMA, AR-GARCH, ARMA-GARCH, and LSTM—using the log-return series of the S&P 500 index under a unified data partition and forecasting strategy. The experimental procedure strictly follows the standard paradigm of time series analysis. Specifically, linear mean models are first constructed on the return scale; residual diagnostics are then conducted to detect conditional heteroskedasticity, which is subsequently addressed through the introduction of GARCH structures; finally, an LSTM model is incorporated as a nonparametric benchmark for comparison under the same forecasting objective.

In the empirical analysis, stationarity of the return series is first confirmed through unit root tests, after which AR and ARMA models are employed to characterize the linear mean dynamics. Ljung–Box test results indicate that these linear models effectively eliminate autocorrelation in the conditional mean, suggesting that they are reasonable for capturing short-term linear dependence in returns. However, subsequent ARCH-LM tests reveal pervasive and statistically significant conditional heteroskedasticity in the residuals, indicating that linear models relying on the constant-variance assumption are insufficient to fully describe the dynamic structure of financial return series.

Motivated by these diagnostic findings, GARCH structures are introduced to model the conditional variance. Estimation results from the AR-GARCH models show that, after incorporating the volatility equation, standardized residuals no longer exhibit significant autocorrelation or remaining ARCH effects. Consequently, the model simultaneously satisfies the key statistical assumptions for both the conditional mean and variance. This result corroborates the classical notion of “volatility clustering” in financial time series theory and highlights the critical role of heteroskedasticity modeling in explaining the structure of return dynamics.

With respect to forecasting performance, in-sample and out-of-sample error metrics for all models are summarized in the corresponding Table 9. The main findings can be summarized as follows:

- AR and ARMA models exhibit favorable in-sample performance according to information criteria, with the ARMA model achieving a slightly lower AIC value than the AR model (−38649.97 versus −38648.03), suggesting that the inclusion of a moving average component marginally improves the fit of the mean structure. However, their out-of-sample forecasting errors are nearly identical, with MSE, RMSE, and MAE all of the same order of magnitude. This indicates that a more complex linear mean specification does not lead to a meaningful improvement in point forecast accuracy for return series.
- After introducing GARCH structures, both AR-GARCH and ARMA-GARCH models achieve a more complete characterization of the joint mean–variance dynamics. Although AIC values for GARCH-type models are not directly comparable to those of pure mean models due to differences in likelihood structure, their out-of-sample MSE, RMSE, and MAE remain nearly identical to those of their corresponding linear counterparts. This finding suggests that the primary advantage of GARCH models lies in capturing volatility dynamics rather than substantially improving point forecast accuracy.
- Comparing AR-GARCH and ARMA-GARCH models further reveals that their forecasting error metrics differ only marginally. However, the ARMA-GARCH model still exhibits some residual autocorrelation, implying that a more complex mean specification does not necessarily enhance overall model quality and may instead introduce parameter redundancy or identification instability.
- As a representative nonparametric approach, the LSTM model does not demonstrate a clear advantage in the same return forecasting task. Its out-of-sample error metrics are slightly higher than those of traditional statistical models, while its AIC value deviates substantially from the scale of linear models, reflecting the limited applicability of information criteria to neural network models. These results suggest that, in noisy daily financial data, complex deep learning models often fail to outperform simpler statistical approaches.
- Overall, differences in out-of-sample forecasting accuracy across models are relatively small. This empirical finding supports a classical view in financial time series theory: the conditional mean of asset returns is only weakly predictable, and improvements in model structure are more effective for explaining volatility characteristics than for achieving substantial gains in point forecast accuracy.

Taken together, the empirical results underscore two key methodological implications. First, residual diagnostics and hypothesis testing of model assumptions are essential steps in ensuring the validity of financial time series models. Second, rather than pursuing model complexity per se, greater emphasis should be placed on constructing models that are theoretically well-motivated, structurally transparent, and well aligned with the underlying data characteristics.

Model	In-Sample	Out-Sample			Key Strength	Key Weakness
	AIC	MSE	RMSE	MAE		
AR(4)	-38648.03	0.000207884	0.014418	0.008986	Simple, interpretable	Ignores volatility
AR-GARCH	-6.5791	0.000208051	0.014424	0.00898	Captures volatility	Slight parameter instability
ARMA(2,1)	-38649.97	0.000207892	0.014418	0.008986	Flexible structure	Parameter redundancy
ARMA-GARCH	-6.5798	0.000208064	0.014424	0.00898	Complete specification	Residual autocorrelation
LSTM	5579.99	0.000210328	0.014503	0.009075	Nonlinear learning	Abnormal AIC

Table 9: Comparative Performance Metrics across All Models

Reference

Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. Journal of econometrics, 31(3), 307-327.

Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. Econometrica: Journal of the econometric society, 987-1007.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural computation, 9(8), 1735-1780.

Wikipedia contributors. (2025, December 17). Occam's razor. In Wikipedia. https://en.wikipedia.org/wiki/Occam%27s_razor

Wikipedia contributors. (2025, December 23). S&P 500. In Wikipedia. https://zh.wikipedia.org/wiki/S%26P_500

Tsay, R. S. (2014). An introduction to analysis of financial data with R. John Wiley & Sons.

Appendix

A. R Code (AR+ARMA+GARCH)

```
# Load package
library(quantmod)
library(tseries)
library(fUnitRoots)
library(forecast)
library(Metrics)
library(FinTS)
library(TSA)
library(rugarch)
# Load data
data=read.table("d-sp58010.txt",header=T)
data_ts <- ts(data$Close)
log_data <- log(data_ts)
diff_data <- diff(log_data) # Log return
# ADF Test
fUnitRoots::adfTest(data_ts)
fUnitRoots::adfTest(log_data)
fUnitRoots::adfTest(diff_data)
# Split data
n <- length(diff_data)
train_size <- floor(0.8 * n)
test_size <- n - train_size
train_data <- diff_data[1:train_size]
test_data <- diff_data[(train_size+1):n]
# 1.AR Model
ar_order <- ar(train_data, method="mle")
ar_model <- arima(train_data, order=c(ar_order$order, 0, 0), include.mean=TRUE)
coefs <- ar_model$coef # Parameter Estimates of AR Model
se <- sqrt(diag(ar_model$var.coef))
t_values <- coefs / se
p_values <- 2 * (1 - pnorm(abs(t_values)))
checkresiduals(ar_model) # Residual diagnosis
# 2.AR-GARCH Model
FinTS::ArchTest(residuals(ar_model)) # ARCH effect test
```

```r
spec_ar <- ugarchspec(variance.model = list(model = "sGARCH", garchOrder = c(1, 1)),
  mean.model = list(armaOrder = c(4, 0), include.mean = TRUE),
  distribution.model = "std")
fit_ar_garch <- ugarchfit(spec = spec_ar, data = train_data, solver = 'hybrid') # Fit
print(fit_ar_garch)
n_ahead <- length(test_data) # Forecast
ar_garch_forecast <- ugarchforecast(fit_ar_garch, n.ahead = n_ahead)
pred_mean_ar_garch <- as.numeric(fitted(ar_garch_forecast))
pred_sigma_ar_garch <- as.numeric(sigma(ar_garch_forecast))
# 3.ARMA Model
arma_model <- auto.arima(train_data, max.p=10, start.p=10, max.q=10, start.q=0,
                         ic="aic", seasonal= FALSE,
                        trace=TRUE,stepwise=TRUE,approximation = FALSE)
coefs <- arma_model$coef # Parameter Estimates of AR Model
se <- sqrt(diag(arma_model$var.coef))
t_values <- coefs / se #
p_values <- 2 * (1 - pnorm(abs(t_values)))
checkresiduals(arma_model) # Residual diagnosis
# 4.ARMA-GARCH Model
FinTS::ArchTest(residuals(arma_model)) # ARCH effect test
spec <- ugarchspec(variance.model = list(model = "sGARCH", garchOrder = c(1, 1)),
  mean.model = list(armaOrder = c(2, 1), include.mean = TRUE),
  distribution.model = "std")
garch_fit_final <- ugarchfit(spec = spec, data = train_data) # Fit
print(garch_fit_final)
n_ahead <- length(test_data)  # Forecast
arma_garch_forecast <- ugarchforecast(garch_fit_final, n.ahead = n_ahead)
pred_mean <- as.numeric(fitted(arma_garch_forecast))
pred_sigma <- as.numeric(sigma(arma_garch_forecast))
# Forecast Performance Metrics
# (1) AR
aic_ar <- AIC(ar_model)
mse_ar <- mean((test_data - ar_pred)^2)
rmse_ar <- sqrt(mse_ar)
mae_ar <- mean(abs(test_data - ar_pred))
# (2) AR-GARCH
info_mat <- infocriteria(fit_ar_garch)
aic_ar_garch <- info_mat[1]
mse_ar_garch <- mean((test_data - pred_mean_ar_garch)^2)
rmse_ar_garch <- sqrt(mse_ar_garch)
mae_ar_garch <- mean(abs(test_data - pred_mean_ar_garch))
# (3) ARMA
aic_arma <- AIC(arma_model)
mse_arma <- mean((test_data - arma_pred)^2)
rmse_arma <- sqrt(mse_arma)
mae_arma <- mean(abs(test_data - arma_pred))
# (4) ARMA-GARCH
info_mat <- infocriteria(garch_fit_final)
aic_value <- info_mat[1]
mse_garch <- mean((test_data - pred_mean)^2)
rmse_garch <- sqrt(mse_garch)
mae_garch <- mean(abs(test_data - pred_mean))
```

```r
# Plot 1
par(mfrow=c(1,2))
plot(data_ts, main="Time Series Diagram: Raw Data", ylab="Closing Price", xlab="Time",
     col="blue", lwd=2)
plot(log_data, main="Time Series Diagram: After Log Transformation", ylab="ln(Closing
Price)", xlab="Time",col="darkgreen", lwd=2)
# Plot 2
acf(data_ts, main="ACF: Raw Data", lag.max=20)
acf(log_data, main="ACF: After Log Transformation", lag.max=20)
# Plot 3 & 4
tsdisplay(diff_data, main="Log Return")
acf(diff_data, main="ACF: Log Return", lag.max=20)
pacf(diff_data, main="PACF: Log Return", lag.max=20)
# Plot 5
qqnorm(ar_model $ residuals,main="QQ Plot: Residuals of AR(4) Model") #QQ plot of AR
residual
qqline(ar_model $ residuals)
# Plot 6
x_axis <- 1:n_ahead
ylim_1 <- range(c(test_data, ar_mean + 2*ar_se, ar_mean - 2*ar_se))
plot(x_axis, as.numeric(test_data), type = "l", col = "gray70", lwd = 0.8, ylim = ylim_1,
main = "AR(4) Model (Full Test Set)", ylab = "Log Returns", xlab = "")
lines(x_axis, ar_mean + 2*ar_se, col = "blue", lty = 2, lwd = 1)
lines(x_axis, ar_mean - 2*ar_se, col = "blue", lty = 2, lwd = 1)
lines(x_axis, ar_mean, col = "red", lwd = 1.2)
legend("topleft", c("Actual", "Mean Forecast", "Static 95% CI"),
       col=c("gray70", "red", "blue"), lty=c(1,1,2), cex=0.7, bty="n")
# Plot 7
ylim_2 <- range(c(test_data, arg_mean + 2*arg_sigma, arg_mean - 2*arg_sigma))
plot(x_axis, as.numeric(test_data), type = "l", col = "gray70", lwd = 0.8, ylim = ylim_2,
main = "AR-GARCH (Full Test Set)", ylab = "Log Returns", xlab = "")
lines(x_axis, arg_mean + 2*arg_sigma, col = "blue", lty = 2, lwd = 0.8)
lines(x_axis, arg_mean - 2*arg_sigma, col = "blue", lty = 2, lwd = 0.8)
lines(x_axis, arg_mean, col = "red", lwd = 1.2)
legend("topleft", c("Actual", "Mean Forecast", "Dynamic GARCH CI"),
       col=c("gray70", "red", "blue"), lty=c(1,1,2), cex=0.7, bty="n")
# Plot 8
qqnorm(arma_model $ residuals,main="QQ Plot: Residuals of ARMA(2,1) Model")
qqline(arma_model $ residuals)
# Plot 9
ylim_3 <- range(c(test_data, arma_mean + 2*arma_se, arma_mean - 2*arma_se))
plot(x_axis, as.numeric(test_data), type = "l", col = "gray70", lwd = 0.8, ylim = ylim_3,
main = "ARMA(2,1) Model (Full Test Set)", ylab = "Log Returns", xlab = "")
lines(x_axis, arma_mean + 2*arma_se, col = "blue", lty = 2, lwd = 1)
lines(x_axis, arma_mean - 2*arma_se, col = "blue", lty = 2, lwd = 1)
lines(x_axis, arma_mean, col = "red", lwd = 1.2)
legend("topleft", c("Actual", "Mean Forecast", "Static 95% CI"),
       col=c("gray70", "red", "blue"), lty=c(1,1,2), cex=0.7, bty="n")
# Plot 10
ylim_4 <- range(c(test_data, armag_mean + 2*armag_sigma, armag_mean - 2*armag_sigma))
plot(x_axis, as.numeric(test_data), type = "l", col = "gray70", lwd = 0.8, ylim =
ylim_4, main = "ARMA-GARCH (Full Test Set)", ylab = "Log Returns", xlab = "")
```

```r
lines(x_axis, armag_mean + 2*armag_sigma, col = "blue", lty = 2, lwd = 0.8)
lines(x_axis, armag_mean - 2*armag_sigma, col = "blue", lty = 2, lwd = 0.8)
lines(x_axis, armag_mean, col = "red", lwd = 1.2)
legend("topleft", c("Actual", "Mean Forecast", "Dynamic GARCH CI"),
       col=c("gray70", "red", "blue"), lty=c(1,1,2), cex=0.7, bty="n")
```

B. Python Code (LSTM Model)

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
import tensorflow as tf
import math

data = pd.read_csv('d-sp58010.txt', sep='\s+')
# date
data['date_str'] = pd.to_datetime({
    'year': data['Year'],
    'month': data['Mon'],
    'day': data['Day']})
data.set_index('date_str', inplace=True)
data.sort_index(inplace=True)
target_col = 'Adjclose'
data['Log_Return'] = np.log(data[target_col] / data[target_col].shift(1))
data.dropna(inplace=True)
# time series data
values = data['Log_Return'].values.reshape(-1, 1)

# Data preprocessing
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(values)

# Define sliding window function
def create_dataset(dataset, look_back=60):
    X, Y = [], []
    for i in range(len(dataset) - look_back):
        X.append(dataset[i:(i + look_back), 0])
        Y.append(dataset[i + look_back, 0])
    return np.array(X), np.array(Y)

look_back = 60
X, Y = create_dataset(scaled_data, look_back)
X = np.reshape(X, (X.shape[0], X.shape[1], 1))

# Split data
train_size = int(len(X) * 0.8)
test_size = len(X) - train_size
X_train, X_test = X[0:train_size], X[train_size:len(X)]
Y_train, Y_test = Y[0:train_size], Y[train_size:len(Y)]
```

```
# Build LSTM Model
model = Sequential()
model.add(LSTM(units=50, return_sequences=True, input_shape=(look_back, 1)))
model.add(Dropout(0.2))
model.add(LSTM(units=50, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(units=1))
model.compile(optimizer='adam', loss='mean_squared_error')

# Train
history = model.fit(X_train, Y_train,
    epochs=10,
    batch_size=32,
    validation_data=(X_test, Y_test),
    verbose=1)

# Predict
train_predict = model.predict(X_train)
test_predict = model.predict(X_test)

# Anti-normalization (Real Log Return)
Y_train_real = scaler.inverse_transform(Y_train.reshape(-1, 1))
train_predict_real = scaler.inverse_transform(train_predict)
Y_test_real = scaler.inverse_transform(Y_test.reshape(-1, 1))
test_predict_real = scaler.inverse_transform(test_predict)

# Metrics
mse_train = mean_squared_error(Y_train_real, train_predict_real)
rmse_train = np.sqrt(mse_train)
mae_train = mean_absolute_error(Y_train_real, train_predict_real)
mse_test = mean_squared_error(Y_test_real, test_predict_real)
rmse_test = np.sqrt(mse_test)
mae_test = mean_absolute_error(Y_test_real, test_predict_real)

# AIC: n * ln(RSS/n) + 2k
n = len(Y_train_real)
k = model.count_params()
resid = Y_train_real - train_predict_real
rss = np.sum(resid ** 2)
if rss > 0:
    aic = n * np.log(rss / n) + 2 * k
else:
    aic = -np.inf

print(f"MSE  : {mse_train:.9f}")
print(f"RMSE : {rmse_train:.9f}")
print(f"MAE  : {mae_train:.9f}")
print(f"AIC  : {aic:.2f}")
print(f"MSE  : {mse_test:.9f}")
print(f"RMSE : {rmse_test:.9f}")
print(f"MAE  : {mae_test:.9f}")
```

```
# Plot
# Loss curve
plt.figure(figsize=(8, 4))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss (Scaled)')
plt.legend()
plt.show()

plt.figure(figsize=(12, 6))
plt.title('Validation Set: Actual vs Predicted Prices', fontsize=14)
plt.plot(Y_test_real, label='Actual Price', linewidth=1.5)
plt.plot(test_predict_real, label='Predicted Price',linewidth=1.5)
plt.legend(fontsize=12)
plt.xlabel('Date')
plt.ylabel('Close Price')
plt.grid(True, alpha=0.3)
plt.show()

plt.figure(figsize=(14, 6))
subset_n = 200
if len(Y_test_real) > subset_n:
    real_data_subset = Y_test_real[-subset_n:]
    pred_data_subset = test_predict_real[-subset_n:]
else:
    real_data_subset = Y_test_real
    pred_data_subset = test_predict_real

time_index = range(len(real_data_subset))
plt.plot(time_index, real_data_subset, color='black', label='Actual Return', alpha=0.6,
linewidth=1)
plt.plot(time_index, pred_data_subset, color='red', label='LSTM Prediction', linewidth=1.5)
plt.title(f'S&P 500 Log Return Prediction (Last {len(real_data_subset)} Days)')
plt.xlabel('Time Steps')
plt.ylabel('Log Return')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```